

Sprawozdanie z projektu: PINN w bibliotece DeepXDE dla problemu Lotka–Volterra

1. Wstęp

W projekcie wykorzystano bibliotekę **DeepXDE** do implementacji sieci neuronowej typu **Physics-Informed Neural Network**, czyli PINN. Jest to metoda uczenia maszynowego, w której sieć neuronowa nie uczy się wyłącznie na podstawie danych pomiarowych, ale dodatkowo uwzględnia równania opisujące dane zjawisko fizyczne, biologiczne lub techniczne.

W klasycznej sieci neuronowej model próbuje jedynie dopasować przewidywania do danych. W przypadku PINN funkcja straty zawiera dodatkowo informację o równaniu różniczkowym. Dzięki temu model nie tylko dopasowuje się do punktów pomiarowych, ale również zachowuje się zgodnie z narzuconym prawem matematycznym.

W projekcie zastosowano backend **PyTorch**, który został ustawiony jako backend obliczeniowy dla DeepXDE. Rozważany problem jest problemem typu **inverse problem**, ponieważ model ma nie tylko wyznaczyć przebieg populacji w czasie, ale również nauczyć się nieznanych parametrów równania Lotka–Volterra.

2. Cel projektu

Celem projektu było zbudowanie modelu PINN dla układu drapieżnik–ofiara na podstawie rzeczywistych danych historycznych dotyczących liczby zajęcy i rysi. W projekcie wykorzystano model Lotka–Volterra, który opisuje zależność między populacją ofiar i drapieżników.

Główne cele projektu:

- wykorzystanie biblioteki DeepXDE do rozwiązania problemu odwrotnego,
- użycie rzeczywistego datasetu hare–lynx,
- dopasowanie modelu do danych pomiarowych,
- uwzględnienie równań różniczkowych w procesie uczenia,
- wyznaczenie gładkich krzywych opisujących populacje zajęcy i rysi,
- analiza wyników na podstawie wykresów.

3. Dataset

W projekcie wykorzystano klasyczny dataset **hare–lynx** pochodzący z danych Hudson Bay Company. Dane obejmują lata **1900–1920** i przedstawiają liczbę pozyskanych skór zajęcy śnieżnych oraz rysi kanadyjskich. Wartości podane są w tysiącach sztuk.

Dane te są często wykorzystywane jako przykład relacji drapieżnik–ofiara. Należy jednak pamiętać, że nie są to bezpośrednie pomiary rzeczywistej populacji zwierząt, lecz liczba pozyskanych skór. Mimo tego przyjmuje się, że liczba skór jest przybliżonym wskaźnikiem zmian populacji w czasie.

Przed treningiem dane zostały znormalizowane. Czas przeskalowano z zakresu lat 1900–1920 do przedziału od 0 do 1. Populacje również zostały przeskalowane, aby ułatwić uczenie sieci neuronowej.

4. Model matematyczny

Do opisu dynamiki populacji wykorzystano klasyczny układ równań Lotka–Volterra:

$$\begin{aligned}\frac{dH}{dt} &= AH - BHL \\ \frac{dL}{dt} &= -CL + DHL\end{aligned}$$

gdzie:

- $H(t)$ oznacza populację zajęcy,
- $L(t)$ oznacza populację rysi,
- A oznacza naturalne tempo wzrostu populacji zajęcy,
- B opisuje wpływ rysi na zmniejszanie populacji zajęcy,
- C oznacza naturalne tempo spadku populacji rysi przy braku pożywienia,
- D opisuje wpływ zajęcy na wzrost populacji rysi.

Znaczenie biologiczne modelu jest następujące: zajęce rozmnażają się naturalnie, ale ich liczba spada w wyniku obecności rysi. Rysie bez pożywienia wymierają, ale gdy dostępna jest duża liczba zajęcy, ich populacja może wzrastać.

5. Metoda PINN

W zastosowanym modelu sieć neuronowa przyjmuje jako wejście czas t , a zwraca dwie wartości:

$$H(t), L(t)$$

czyli przewidywaną liczbę zajęcy oraz rysi w danym czasie.

DeepXDE automatycznie oblicza pochodne przewidywanych funkcji względem czasu, czyli:

$$\frac{dH}{dt}, \frac{dL}{dt}$$

Następnie sprawdzane jest, czy przewidywane funkcje spełniają równania Lotka–Volterra. Jeżeli równanie jest spełnione, residual równania jest bliski zeru.

Funkcja straty w projekcie składa się z dwóch głównych części:

$$Loss = Loss_{data} + Loss_{ODE}$$

gdzie:

- $Loss_{data}$ oznacza błąd dopasowania modelu do rzeczywistych danych,
- $Loss_{ODE}$ oznacza błąd wynikający z niespełnienia równań różniczkowych.

Dzięki temu model nie jest zwykłą regresją. Sieć neuronowa jednocześnie dopasowuje się do danych i zachowuje zgodność z modelem biologicznym.

6. Implementacja w DeepXDE

Projekt zaimplementowano w języku Python z użyciem biblioteki DeepXDE. Backend obliczeniowy ustawiono na PyTorch:

$$DDE_BACKEND = \text{pytorch}$$

Dane pomiarowe zostały dodane do modelu za pomocą `PointSetBC`, osobno dla populacji zajęcy oraz rysy. Model zbudowano jako sieć typu `FNN`, czyli fully connected neural network.

Nieznane parametry równania A, B, C, D zostały zdefiniowane jako `dde.Variable`. Oznacza to, że są one trenowane razem z wagami sieci neuronowej. Z tego powodu projekt jest przykładem problemu odwrotnego, ponieważ parametry modelu są wyznaczone na podstawie danych.

W treningu zastosowano dwa optymalizatory. Najpierw użyto optymalizatora Adam, który pozwala znaleźć przybliżone rozwiązanie. Następnie zastosowano L-BFGS, który służy do dokładniejszego dopracowania wyniku.

7. Wyniki

Pierwszy wykres przedstawia rzeczywiste dane hare-lynx. Na osi poziomej znajduje się rok, a na osi pionowej liczba skór w tysiącach. Punkty niebieskie oznaczają dane dla zajęcy, a punkty pomarańczowe dane dla rysy. Na wykresie widoczna jest cykliczna zależność między populacjami. Najpierw rośnie liczba zajęcy, a następnie z pewnym opóźnieniem rośnie liczba rysy.

Drugi wykres przedstawia porównanie danych rzeczywistych z predykcją modelu PINN. Punkty oznaczają dane pomiarowe, natomiast linie przedstawiają przewidywania sieci. Model dobrze odtwarza ogólny cykliczny charakter zmian populacji. Krzywe nie muszą idealnie przechodzić przez każdy punkt, ponieważ model jest ograniczony nie tylko danymi, ale również równaniem różniczkowym Lotka-Volterra.

Trzeci wykres to wykres fazowy, na którym oś pozioma przedstawia liczbę zajęcy, a oś pionowa liczbę rysy. Trajektoria tworzy pętlę, co jest charakterystyczne dla układu drapieżnik-ofiara. Wykres ten pokazuje, że wzrost jednej populacji wpływa z opóźnieniem na drugą populację.

Czwarty wykres przedstawia historię funkcji straty. Spadek wartości loss oznacza, że model uczył się poprawnie. Widoczny spadek funkcji straty oznacza, że model coraz lepiej dopasowywał się do danych oraz coraz lepiej spełniał równania Lotka-Volterra. Silniejszy spadek pod koniec treningu można powiązać z zastosowaniem optymalizatora L-BFGS.

8. Interpretacja wyników

Uzyskane wyniki pokazują, że PINN jest w stanie połączyć dane pomiarowe z równaniem różniczkowym opisującym zjawisko. Model nauczył się gładkich funkcji opisujących zmiany liczby zajęcy i rysi w czasie. Jednocześnie zachował charakterystyczną cykliczną dynamikę układu predator–prey.

Najważniejszą zaletą zastosowanej metody jest to, że model nie działa jak zwykła sieć neuronowa dopasowująca się tylko do punktów. PINN wykorzystuje dodatkową wiedzę w postaci równania Lotka–Volterra, dzięki czemu uzyskane rozwiązanie ma większy sens biologiczny.

9. Ograniczenia projektu

Model Lotka–Volterra jest uproszczeniem rzeczywistości. Nie uwzględnia wielu czynników, takich jak zmiany środowiskowe, choroby, migracje, wpływ człowieka, klimat czy zmienność dostępności pożywienia. Dodatkowo wykorzystany dataset nie opisuje bezpośrednio rzeczywistej liczby zwierząt, ale liczbę pozyskanych skór.

Z tego powodu celem projektu nie było idealne dopasowanie każdego punktu danych, lecz pokazanie, że metoda PINN potrafi połączyć dane historyczne z modelem matematycznym i odtworzyć ogólną dynamikę układu.

10. Wnioski

W projekcie udało się zaimplementować model Physics-Informed Neural Network w bibliotece DeepXDE z backendem PyTorch. Wykorzystano rzeczywiste dane hare–lynx oraz model Lotka–Volterra opisujący relację drapieżnik–ofiara. Model został sformułowany jako inverse problem, ponieważ parametry równań były uczone na podstawie danych.

Wyniki pokazują, że PINN potrafi odtworzyć cykliczny charakter populacji zajęcy i rysi. Wykresy potwierdzają, że model dopasował się do danych oraz zachował strukturę dynamiczną charakterystyczną dla układu predator–prey. Projekt pokazuje praktyczne zastosowanie DeepXDE do problemów odwrotnych opisanych równaniami różniczkowymi.